



CosmWasm Developer's Guide: From Zero to Hero (The DTC Way)

Écrit pour Monkey & la Dream Truth Community

Table des Matières

1. [Introduction: Pourquoi CosmWasm?](#)
 2. [Partie I: Lancer une Blockchain Nickel](#)
 3. [Partie II: Comment Ça Marche? \(La Magie Derrière le Rideau\)](#)
 4. [Partie III: Développer Votre Premier Smart Contract](#)
 5. [Partie IV: Mettre en Pause et Relancer](#)
 6. [Cheat Sheet: Les Commandes Essentielles](#)
-

Introduction: Pourquoi CosmWasm?

Bienvenue dans le monde des **blockchains intelligentes**. Si vous lisez ceci, c'est que vous avez probablement passé les 3 dernières heures à combattre une "boucle infinie" (oui, nous savons, c'est infernal). Mais rassurez-vous : **vous avez vaincu le chaos**. Votre blockchain CosmWasm tourne maintenant, et c'est le moment de faire des choses vraiment cool.

Qu'est-ce que CosmWasm?

CosmWasm est un **moteur de Smart Contracts** pour la blockchain Cosmos. Pensez-y comme ceci :

- **Cosmos** = La chaîne de blocs (le serveur)
- **CosmWasm** = Le langage pour écrire des contrats intelligents (le code métier)

- **Vous** = Le développeur qui va changer le monde (ou au moins, créer un compteur qui fonctionne)

Pourquoi c'est cool pour la DTC?

Pour votre MVP MRV & Ops Copilot, CosmWasm vous permet de :

- **Stocker des événements** immuablement sur la blockchain (pas de triche, pas de "oups, j'ai supprimé la base de données")
- **Calculer des scores Karma** de manière transparente et vérifiable
- **Créer des contrats de gouvernance** pour que la DTC soit vraiment décentralisée
- **Intégrer avec d'autres chaînes** via IBC (Inter-Blockchain Communication)

C'est la fondation de votre "single source of truth".

Partie I: Lancer une Blockchain Nickel

Prérequis

Avant de commencer, assurez-vous que vous avez :

- **WSL 2** (ou Linux) avec un terminal fonctionnel
- **wasmd v0.53.2** (ou compatible) installé
- **Rust** et **cargo** (pour compiler les contrats)
- **Docker** (pour l'optimisation des contrats)

La Séquence Magique (Copier-Coller Friendly)

Voici la **vraie** méthode qui fonctionne. Pas de contournements, pas de patches jq bizarres. Juste la bonne vieille rigueur.

Étape 1: Nettoyage Complet

```
HOME_DIR="$HOME/.wasmd"
CHAIN_ID="localnet"
MONIKER="mynode"
KEYRING_BACKEND="test"

wasmd tendermint unsafe-reset-all --home "$HOME_DIR"
rm -rf "$HOME_DIR/config" "$HOME_DIR/data"
rm -f my_wallet.json
```

Étape 2: Initialisation

```
wasmd init "$MONIKER" --chain-id "$CHAIN_ID" --home "$HOME_DIR"
```

Étape 3: Créer Votre Portefeuille

```
wasmd keys delete my-wallet --keyring-backend "$KEYRING_BACKEND" --home
"$HOME_DIR" -y 2>/dev/null

wasmd keys add my-wallet --keyring-backend "$KEYRING_BACKEND" --home
"$HOME_DIR" --output json > my_wallet.json 2>/dev/null

WALLET_ADDRESS=$(jq -r '.address' my_wallet.json)
echo "Votre adresse: $WALLET_ADDRESS"
```

Étape 4: Financer Votre Portefeuille

```
wasmd genesis add-genesis-account "$WALLET_ADDRESS" 2000000000000000stake \
--home "$HOME_DIR" --keyring-backend "$KEYRING_BACKEND"
```

Étape 5: Devenir Valideur

```
wasmd genesis gentx my-wallet 1000000stake \
--chain-id "$CHAIN_ID" \
--keyring-backend "$KEYRING_BACKEND" \
--home "$HOME_DIR"

wasmd genesis collect-gentxs --home "$HOME_DIR"
```

Étape 6: Optimiser la Configuration

```
CONFIG_PATH="$HOME_DIR/config/config.toml"

sed -i 's/addr_book_strict = true/addr_book_strict = false/g' "$CONFIG_PATH"
sed -i 's/pex = true/pex = false/g' "$CONFIG_PATH"
sed -i 's/allow_duplicate_ip = false/allow_duplicate_ip = true/g'
"$CONFIG_PATH"
sed -i 's/timeout_commit = "5s"/timeout_commit = "1s"/g' "$CONFIG_PATH"
```

Étape 7: Valider et Démarrer

```
wasmd genesis validate-genesis --home "$HOME_DIR"  
wasmd start --home "$HOME_DIR"
```

Partie II: Comment Ça Marche?

L'Architecture

Une blockchain Cosmos fonctionne comme une **usine de production de blocs** :

- **Cosmos SDK** = La couche applicative (gère les comptes, les transactions, les modules)
- **CometBFT** = Le moteur de consensus (produit les blocs, valide les transactions)
- **CosmWasm** = Le module pour les Smart Contracts

Le genesis.json

C'est le **livre de loi initial** qui définit :

- Les comptes initiaux et leurs soldes
- Les validateurs et leur puissance
- Les paramètres de la chaîne

Le Cycle de Vie d'une Transaction

1. Vous créez une transaction
2. Vous la signez avec votre clé privée
3. Vous l'envoyez au nœud
4. Le nœud la valide
5. Le nœud l'inclut dans le bloc suivant
6. CometBFT signe le bloc
7. Le bloc est ajouté à la chaîne

Pourquoi la "Boucle Infinie"?

Les causes courantes :

1. **Désalignement des clés** : Le validateur dans `genesis.json` ne correspond pas à la clé réelle
 2. **Solution** : Utiliser `gentx` pour synchroniser automatiquement
 3. **Masse monétaire incorrecte** : Le total des jetons ne correspond pas
 4. **Solution** : Laisser `collect-gentxs` corriger automatiquement
 5. **Timeouts trop longs** : `timeout_commit = "5s"` ralentit la production
 6. **Solution** : Réduire à `"1s"` pour le développement
 7. **Adresses non routables** : `addr_book_strict = true` rejette les adresses locales
 8. **Solution** : Mettre à `false`
-

Partie III: Développer Votre Premier Smart Contract

Prérequis

```
curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh
source $HOME/.cargo/env
cargo install cargo-generate
docker --version
```

Créer et Compiler

```
cargo generate --git https://github.com/CosmWasm/cw-template.git --name hello-world
cd hello-world

docker run --rm -v "$(pwd)":/code \
  --mount type=volume,source="$`(basename "$(pwd)")_cache",target=/code/target \
  --mount type=volume,source=registry_cache,target=/usr/local/cargo/registry \
  cosmwasm/rust-optimizer:0.12.6
```

Déployer (Terminal 2)

```
WALLET_ADDRESS=$(wasmd keys show my-wallet -a --keyring-backend test)

# Uploader le contrat
wasmd tx wasm store target/wasm32-unknown-unknown/release/hello_world.wasm \
  --from my-wallet \
  --gas-prices 0.025stake \
  --gas auto \
  --chain-id localnet \
  --keyring-backend test \
  --node tcp://localhost:26657 \
  -y

# Récupérer le code ID
CODE_ID=$(wasmd query wasm list-code --node tcp://localhost:26657 --output json
| jq '.code_infos[-1].code_id')

# Instancier
wasmd tx wasm instantiate "$CODE_ID" '{}' \
  --from my-wallet \
  --label "hello-world-instance" \
  --admin "$WALLET_ADDRESS" \
  --gas-prices 0.025stake \
  --gas auto \
  --chain-id localnet \
  --keyring-backend test \
  --node tcp://localhost:26657 \
  -y

# Récupérer l'adresse du contrat
CONTRACT_ADDR=$(wasmd query wasm list-contract-by-code "`$CODE_ID" \
  --node tcp://localhost:26657 \
  --output json | jq -r '.contracts[0]')
```

Interroger et Exécuter

```
# Lire l'état
wasmd query wasm contract-state smart "$CONTRACT_ADDR" '{"get_count":{}}' \
  --node tcp://localhost:26657 \
  --output json

# Incrémenter
wasmd tx wasm execute "$CONTRACT_ADDR" '{"increment":{}}' \
  --from my-wallet \
  --gas-prices 0.025stake \
  --gas auto \
  --chain-id localnet \
  --keyring-backend test \
  --node tcp://localhost:26657 \
  -y

# Vérifier le changement
wasmd query wasm contract-state smart "$CONTRACT_ADDR" '{"get_count":{}}' \
  --node tcp://localhost:26657 \
  --output json
```

Partie IV: Mettre en Pause et Relancer

Arrêter

```
# Si au premier plan
Ctrl + C

# Si en arrière-plan
killall wasmd
```

Redémarrer

```
wasmd start --home ~/.wasmd
```

Votre blockchain reprend exactement là où elle s'est arrêtée!

Cheat Sheet

Gestion du Nœud

Action	Commande
Démarrer	<code>wasmd start --home ~/.wasmd</code>
Arrêter	<code>Ctrl + C</code> ou <code>killall wasmd</code>
Vérifier l'état	<code>wasmd status --node tcp://localhost:26657</code>
Réinitialiser	<code>wasmd tendermint unsafe-reset-all --home ~/.wasmd</code>

Gestion des Clés

Action	Commande
Créer	<code>wasmd keys add my-wallet --keyring-backend test</code>
Lister	<code>wasmd keys list --keyring-backend test</code>
Voir l'adresse	<code>wasmd keys show my-wallet -a --keyring-backend test</code>
Supprimer	<code>wasmd keys delete my-wallet --keyring-backend test -y</code>

Contrats

Action	Commande
Compiler	<code>docker run --rm -v "\$(pwd)":/code ... cosmwasm/rust-optimizer:0.12.6</code>
Uploader	<code>wasmd tx wasm store hello_world.wasm --from my-wallet ...</code>
Instancier	<code>wasmd tx wasm instantiate [CODE_ID] '{}' --from my-wallet ...</code>
Interroger	<code>wasmd query wasm contract-state smart [ADDR] '{"get_count":{}}' ...</code>
Exécuter	<code>wasmd tx wasm execute [ADDR] '{"increment":{}}' --from my-wallet ...</code>

Conclusion

Vous avez maintenant :

✅ Lancé une blockchain CosmWasm entièrement fonctionnelle ✅ Compris comment le consensus fonctionne ✅ Déployé et testé un Smart Contract ✅ Interagi avec votre contrat

Prochaines étapes pour la DTC:

1. Créer un contrat pour stocker les événements (MVP MRV & Ops Copilot)
2. Implémenter le calcul du Karma en Rust/CosmWasm
3. Ajouter la gouvernance (votes, propositions)

4. Connecter avec DuckDB et Metabase
5. Lancer sur testnet Cosmos
6. Passer en production

Vous avez les outils. Maintenant, **construisez l'avenir de la DTC!** 🚀

Écrit avec ❤️ par Manus AI pour Monkey & la Dream Truth Community

"Parce que la vérité doit être vérifiable, et le code est la loi."